Machine Learning Final Project:
CATEGORIZING CYBER SECURITY ATTACKS ON IOT INFRASTRUCTURE

Andi (B. A.) RODRÍGUEZ MACUACÉ

Teacher

Christophe RODRIGUES

Pole Universitaire Leonard de Vinci - ESILV

Cybersecurity & Cloud Computing

Machine Learning

Paris - France

Dec 2025

Table of Contents

1. Business Case

As Internet of Things (IoT) technologies continue to be more prevalent in our day to day due to enhanced technology, increased access to the Internet, and the generation of big data (Sharma y cols., 2025) and hence, the number of connected devices has grown significantly in proportion to their massive adoption. The rise of IoT networks has introduced scalable but highly vulnerable infrastructures (Musthafa y cols., 2024). This large-scale connectivity introduces significant cyber security risks, since these devices often operate with minimal built-in security mechanisms, and this in combination with their large scale connectivity, results in susceptible devices, vulnerable to exploitation by malicious actors who can potentially perform network intrusions, denial of service attacks, and large-scale botnet operations. This problem leads to an urgent need for reliable intrusion detection systems (Musthafa y cols., 2024).

One of the big challenges in attempting to secure IoT environments lies in the lack of visibility over continuous and high volume network traffic. Traditional cyber attack detection systems struggle to analyse traffic in real-time at the scale required for modern networks. Therefore, automated and intelligent approaches become a better alternative and rather essential for detecting abnormal behavior accurately and in a short amount of time. To achieve this, Selecting the proper data features for effectively training such ML models is critical to maximizing detection accuracy and computational efficiency (Samantaray, Barik, y Biswal, 2024).

The purpose of this project is to build a supervised Machine Learning model that enables the classification of a variety of network flows into distinct cyber attack types, with the use of features extracted from real IoT infrastructure found in Kaggle. By automatically identifying malicious traffic, this model could support faster incident response, reduce the risk of network compromise, and ultimately contribute to stronger IoT network monitoring strategies.

Github Repository link: *"https://github.com/Andriguez/Machine-Learning-Project".*

2. Problem Formalization and Objectives

As previously remarked, the objective of this project is to design a Machine Learning model for intrusion detection, that i s capable of classifying malicious network traffic by specific cyber attack categories. The problem is therefore framed as a *supervised multiclass classification task*, where each network flow is described by a collection of numerical and categorical features extracted from real-time IoT network communication.

Formally, each network flow instance is defined as a feature vector:

$$x_i = [f_1, f_2, ..., f_n] \in \mathbb{R}^n$$

where f_j represents a traffic-related attribute such as packet rate, flow duration, byte count, or protocol information. Each instance is associated with a label y_i selected from one of K possible attack types:

$$y_i \in \{1, 2, ..., K\}$$

The objective is to learn a function:

$$\hat{y} = h(x) \quad \text{where } h : \mathbb{R}^n \to \{1, ..., K\}$$

that minimizes the classification error on unseen network traffic.

Because the dataset contains a strong class imbalance between attack types, the model must also maintain accurate performance across all classes, instead of being biased towards the majority class. For this reason, the model evaluation relies on multiple metrics sensitive to imbalance, such as Precision, Recall, F1-score, confusion matrices, and ROC-AUC for multiclass classification.

Furthermore, the model must achieve three key goals:

- **Accuracy** by Correctly classifying the majority of flows.
- **Generalization** by Performing well on unseen data while avoiding overfitting.
- **Interpretability** by Providing key insights into which features contribute most to accurate classification

This formalization section provides the foundation for the methodological and experimental decisions further described in the upcoming chapters.

2.1. Algorithm description

During the machine learning labs, we used different models for specific use cases, which served as an introduction to their implementation and gave us a general idea on which could be used to solve the problem presented in our projects, after further research and in order to explore different learning capabilities, as well as robustness levels, multiple supervised algorithms were chosen to be trained, fitted and predicted with. Each model introduces different learning assumptions and decision boundaries, allowing a more complete evaluation of classification performance in the given context:

- The **Logistic Regression** model is a linear decision model that provides a strong baseline due to its speed, low computational cost, and interpretability. It is a popular choice when dealing with high dimensional data and can easily serve as a reference point for more complex models.

- The **Random Forest Classifier** model is a tree-based ensemble method capable of capturing non-linear and complex feature interactions, while simultaneously providing importance scores for the features. It does not get majorly affected by noise and reduces overfitting by aggregating multiple decision trees.

- The **Support Vector Machine (SVM)** is a powerful classifier that seeks optimal decision hyperplanes, especially effective in high dimensional datasets. It classifies data by defining a collection of support vectors (Almaiah y cols., 2022) and the kernel trick allows learning complex non-linear boundaries between attack classes.

- The **Voting Ensemble Classifier** is a hybrid model that combines predictions from the three previously mentioned models using soft-voting. This approach enables us to combine the complementary strengths of each model, to achieve overall higher predictive performance and stability.

2.2. Limitations of Approaches

While the selected models provide strong baselines for intrusion detection, they also come with specific constraints, for example:

- Logistic Regression is limited by its linear decision boundary, making it less effective on complex decision regions such as the network traffic found in the dataset.

- Random Forest may become computationally intensive with large feature spaces and might still struggle with extremely rare attack types.
- SVM's performance strongly depends on parameter selection and scaling. It can be slow for very large datasets, especially when using non-linear kernels.
- Voting Ensembles dramatically increase model complexity and can decrease interpretability despite their more accurate performance.

Additionally, the presence of severe class imbalance in the dataset can cause sensitivity to majority classes, for which specific techniques might have to be used to avoid bias. At the same time, oversampling can also introduce synthetic noise and influence how well models generalize real world attacks.

These limitations shape the methodological strategy, influencing specific decisions made in the code to balance performance improvements and computational cost, as well as interpretability.

3. Methodology

In this chapter, the methodology used to solve the classification problem will be explained in detail, providing a step by step guide of how the dataset was examined and pre-processed; as well as how the models were developed and implemented to ensure the accurate categorization of attacks in IoT. The notebook described in this project report and the dataset can both be found in the github repository *"Machine-Learning-Project"*.

3.1. Dataset Description and Exploration

The dataset *RT-IoT2022* used in this project was found in *Kaggle* (*Real-Time IoT2022 Cyber Attack Dataset*, s.f.) uploaded in 2023. It is a proprietary dataset collected from a real IoT network infrastructure and includes both normal and malicious traffic flows representing real world scenarios.

The network traffic was collected from multiple IoT devices such as *ThingSpeak-LED*, *Wipro-Bulb* and *MQTT-Temp*, while cyber-attack scenarios were simulated using techniques like *Brute-Force SSH* and *DDoS floods*. These connections were monitored and processed in order to obtain specific network metadata, in order to facilitate categorization through a wide range of features, making it a valuable resource for developing and evaluating intrusion detection systems, such as the one proposed in this project.

RT-IoT2022 contains a total of 85 columns, most of them numerical values describing specific network metadata and 3 text based categories (proto, service and attack_type); and it has a total of *123.117* entries of which none were found to have any missing values.

The *Attack_type* column serves as the target feature, and it contains a total of 12 categories for attacks: *DOS_SYN_Hping*, *ARP_poisioning*, *NMAP_UDP_SCAN*, *NMAP_XMAS_TREE_SCAN*, *NMAP_OS_DETECTION*, *NMAP_TCP_scan*, *DDOS_Slowloris*, *Metasploit_Brute_Force_SSH*, *NMAP_FIN_SCAN*, *MQTT*, *Thing_speak* and *Wipro_bulb_Dataset*

3.1.1. Statistical Summary

A descriptive statistics analysis was performed on the 82 numeric features derived from the dataset. The summary indicates a wide variation in scale across the different features, highlighting the diverse nature of IoT network traffic.

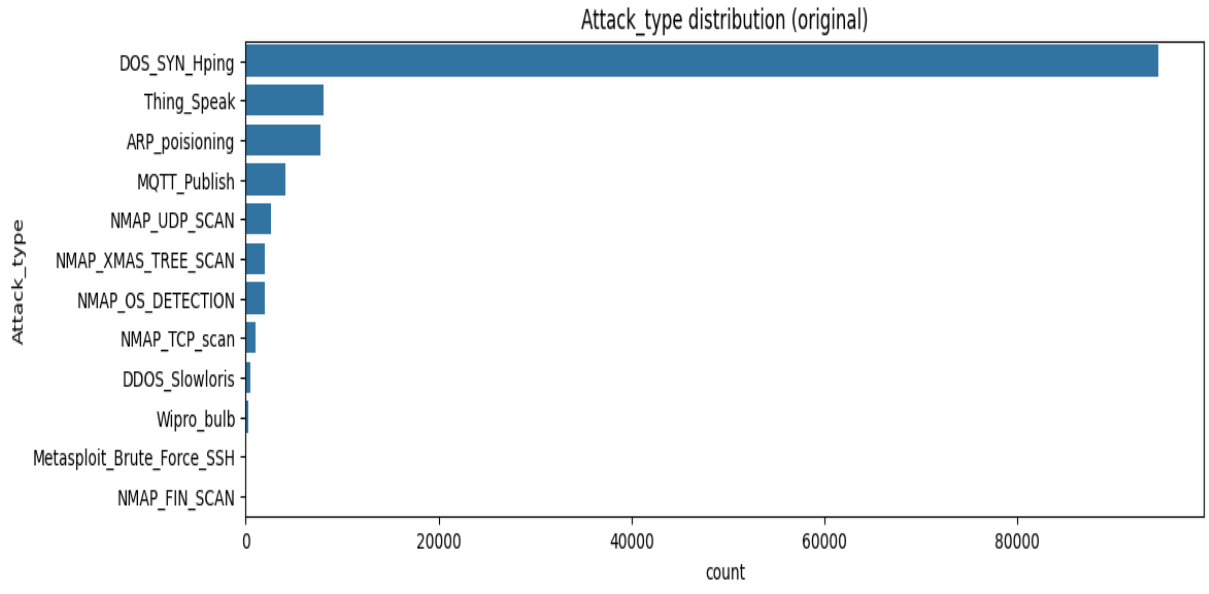
Several attributes like packet counts, flow duration or idle time showed highly skewed distributions. Additionally, for many features, the median and upper quartiles are equal to zero, while the

maximum values are extremely high, suggesting that most IoT flows are small and short lived but on the other hand, a minority correspond to abnormal network behavior, most likely associated with malicious activities.

Furthermore, large standard deviations relative to the mean confirm the presence of heavy-tailed behavior, which is typical in this type of datasets, where attacks generate bursts of packets or long duration flows. These statistical properties support the need for scaling and handling imbalance during the pre-processing stage since learning algorithms could otherwise become bias towards more common network flows.

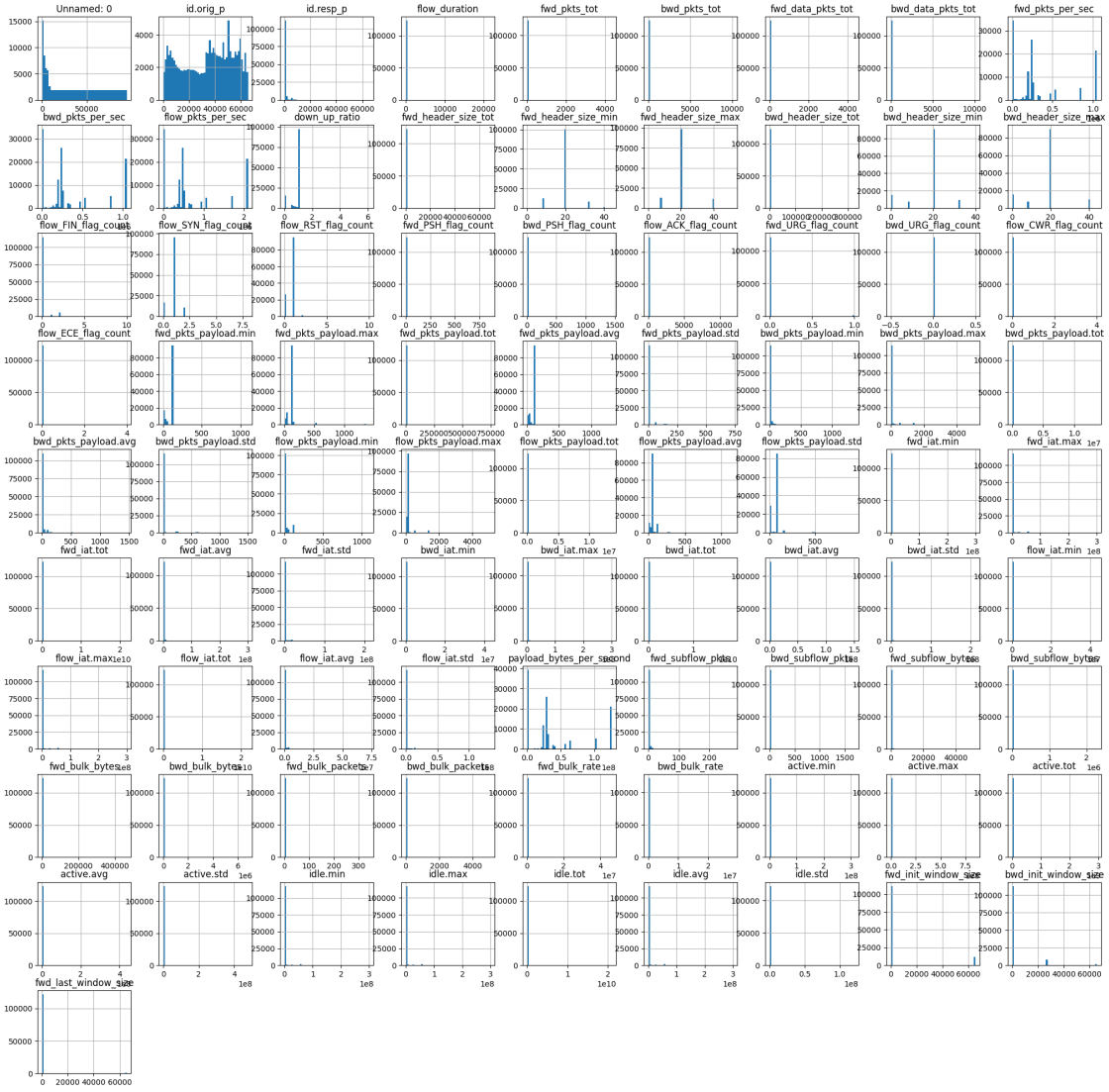
3.1.2. Class Distribution and Imbalance Visualization

A class distribution was plotted for the dataset target *Attack_type* in order to clearly visualize how balanced the categories were, The bar chart bellow shows a severe imbalance in class occurrences like *DOS_SYN_Hping* which contained a total of 94.659 samples, while others like *Metasploit_Brute_Force_SSH* or *NMAP_TCP_scan* had less than 40 samples each. This demonstrated a very severe class imbalance which could affect the prediction as models trained on the raw dataset would most likely achieve high accuracy levels by predicting mostly the majority class while failing to learn minority attack classes. For this reason it was decided to incorporate the handling of class imbalance into the pre-processing stage to.



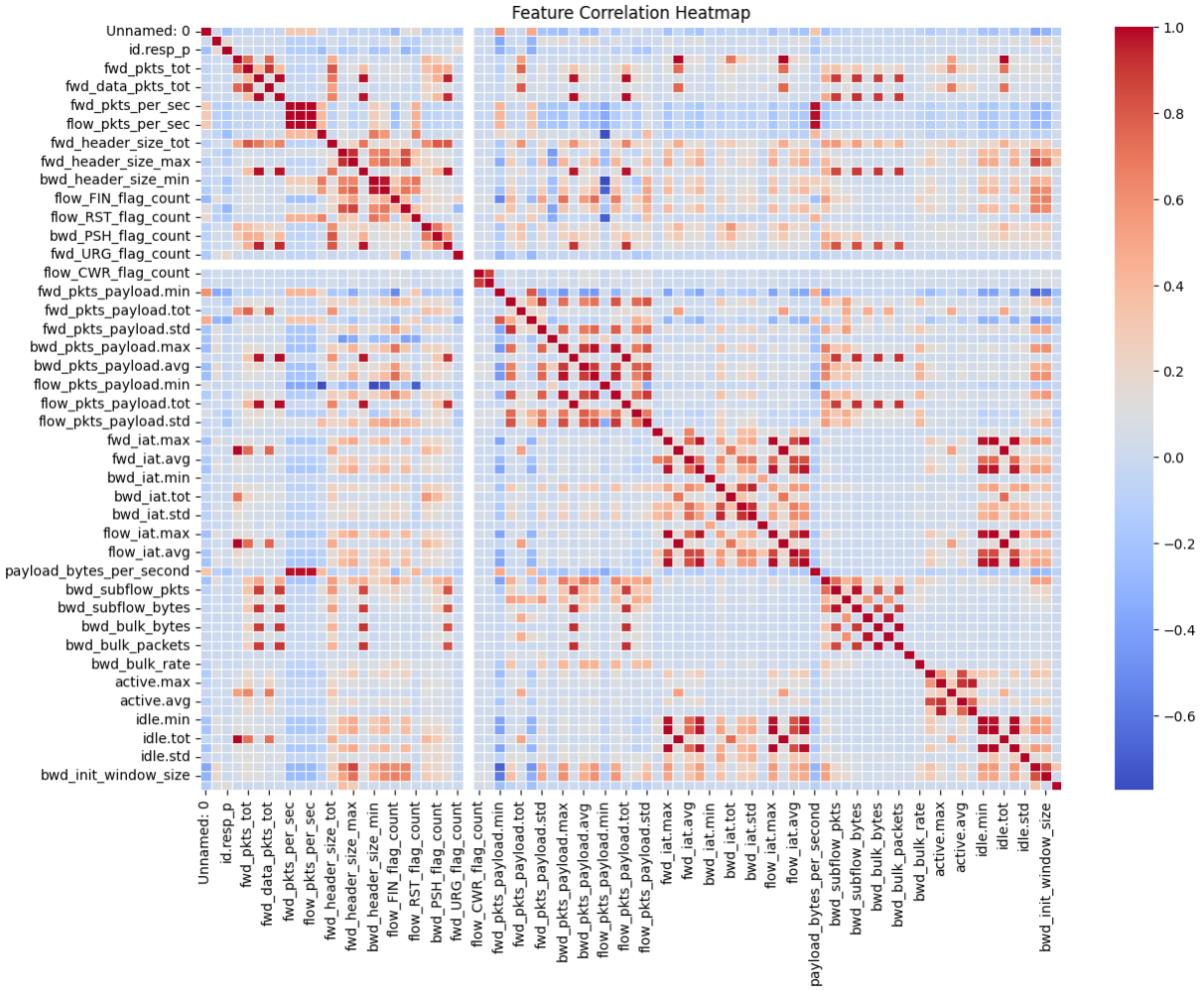
3.1.3. Feature Distribution Analysis

During the data analysis, the feature distribution of the dataset was plotted to understand data characteristics like the spread of the feature's values, as well as to identify any possible patterns found in the raw data. The histogram grid below shows that most features are very right skewed and heavily tailed, with a big spike near zero and a long tail for bigger values, making the dataset not normally distributed, on the other hand, Several TCP-flag features such as *fwd_FIN_flag_count* and *bwd_RST_flag_count* show mostly zero values, indicating that these events are infrequent and almost binary, carrying information only when the events occur. The overall distributions are asymmetric and multi-modal. Overall, the histograms show that normal and malicious traffic do not follow similar statistical patterns, in addition to demonstrating that certain features seem to be influenced by specific attack mechanisms. Lastly, this inspection served as a justification to include standardization and the selective removal of low variance features in the dataset.



3.1.4. Correlation Analysis

A correlation matrix was plotted as part of the analysis in order to understand which features in the dataset had a stronger relationship, as well as enabling us to visualize patterns and relationships between them. The heatmap revealed highly correlated feature groups, indicating multiple features captured the same underlying network behavior, suggesting high redundancy and potential risk of *multicollinearity* in linear models such as *Logistic Regression*, potentially reducing model stability and interpretability. These findings further justified the implementation of dimensionality reduction using PCA to reduce highly redundant information and the use of GridSearchCV for hyperparameter tuning of the C parameter in the base model; as well as the decision to evaluate with the *Random Forest* model which can better handle correlation.

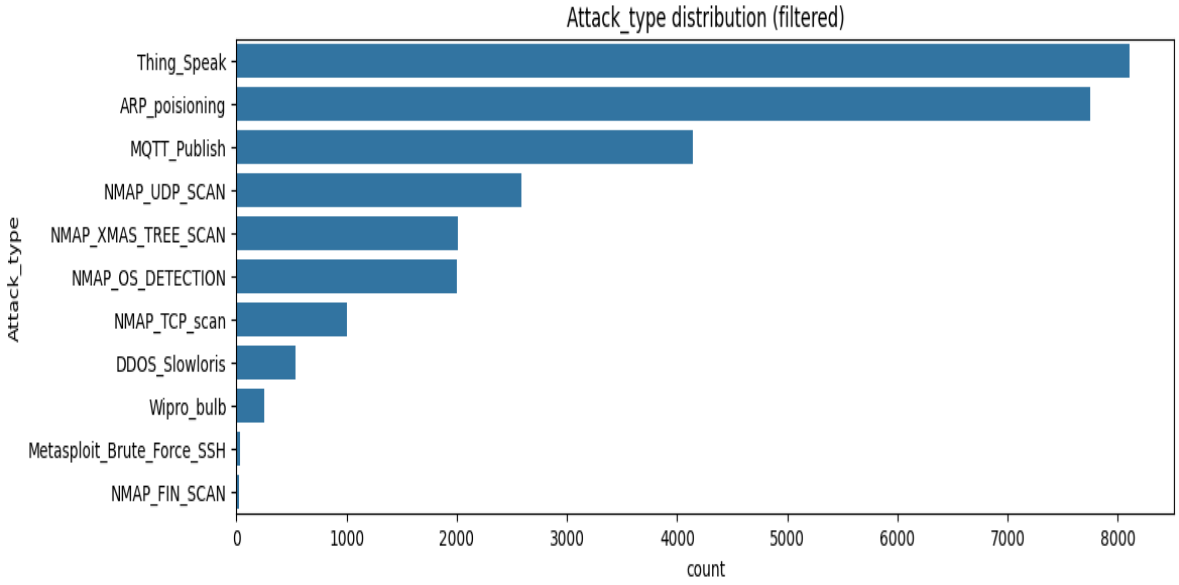


3.2. Data Pre-processing

Reliable model performance requires that raw data undergoes extensive validation and transformation to ensure quality, consistency, and suitability for machine learning models. For this reason, after inspecting and analyzing the dataset, a series of steps were applied to the dataset as part of the pre-processing stage later detailed in this chapter.

3.2.1. Initial Cleaning

After plotting the class distribution during the data exploration stage, it was found that the *DOS_SYN_Hping* class had far more samples than all others, causing a severe class imbalance and potentially affecting the prediction by creating misleading accuracy and preventing the mode from learning other attacks. For this reason, as the class distribution below shows, the *DOS_SYN_Hping* category was removed from the dataset. Additionally, as it is custom in the pre-processing of data for machine learning models, identifier columns such as *id.orig_p* and *id.resp_p* were removed to prevent overfitting and to avoid these features influencing the model.



As shown in the plotted class distribution of the filtered target, the class imbalance remains even after removing the majority class but to a much lesser level, in order to fix this and ensure accurate anomaly detection in IoT networks, where malicious activities may be underrepresented in the data (Musthafa y cols., 2024), SMOTE was used as an extra step on the pre-processing stage.

3.2.2. Feature Redundancy Reduction via Correlation

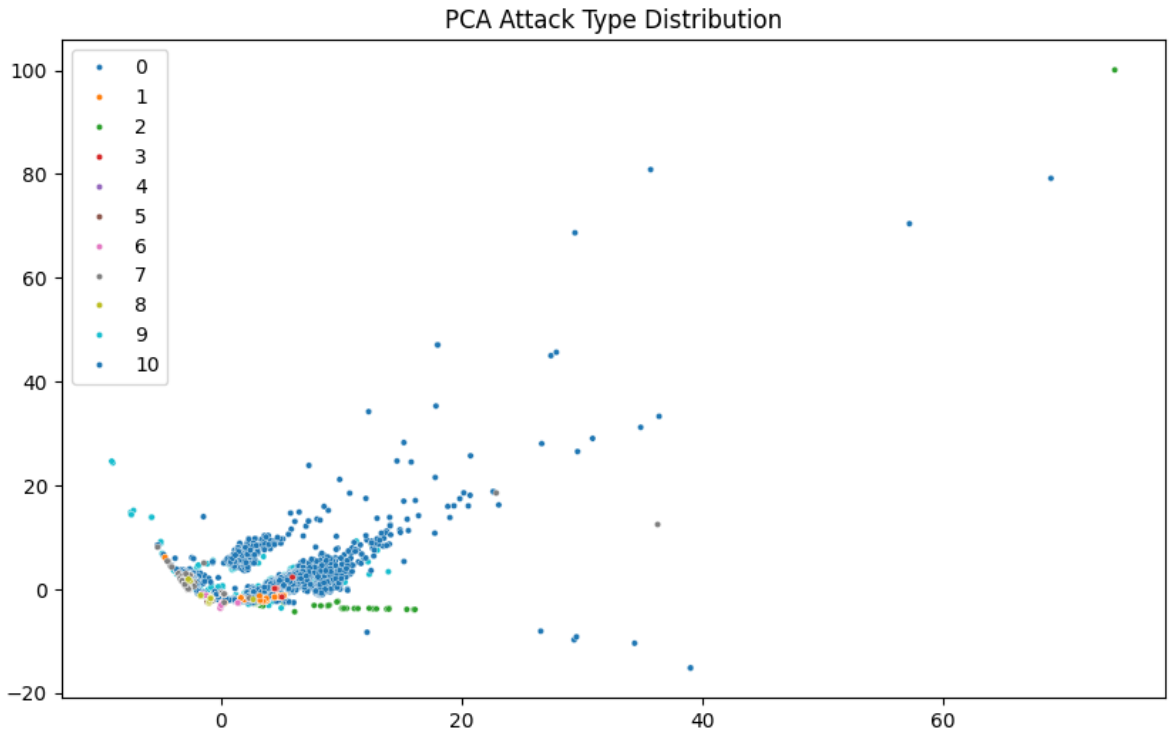
After plotting the correlation heat map during the data exploration phase, it was found that multiple features captured the same network behavior, in an attempt to reduce multicollinearity (mainly for the base model) and improve interpretability, correlation was computed on the numerical features, and based on this operation (correlation >0.95), 26 features that carried nearly identical information were removed from the database, leaving a total of 56 features left.

3.2.3. Encoding Categorical Features

As mentioned previously, the dataset contained a total of 3 text-based features, namely *proto*, *service* , and *Attack_type*. Because machine learning algorithms require numerical input, a function was coded and executed to convert these categorical features into integers using Label Encoding, which allows models to process them, as ML and DL only work with numerical values (Musthafa y cols., 2024), while maintaining the unique categories without losing any information. What to explain

3.2.4. PCA-based Feature Space Exploration

During one of the project review sessions with the teacher, it was recommended to apply PCA to the dataset with the purpose of showing how attacks are spread in the feature space and whether their separation was possible. Following the recommendation, an initial PCA was applied only for visualization purposes. The initial plot displayed below, showed that some classes are spread over a huge region, dominating the feature space while minority classes appear to be very close together like classes 3 and 5, there is a clear overlap between different groups, indicating non-linear decisions boundaries which makes classification challenging for the base model. This supports the decision of training the more advanced models (*SVM* and *Random Forest*) along with the baseline model.



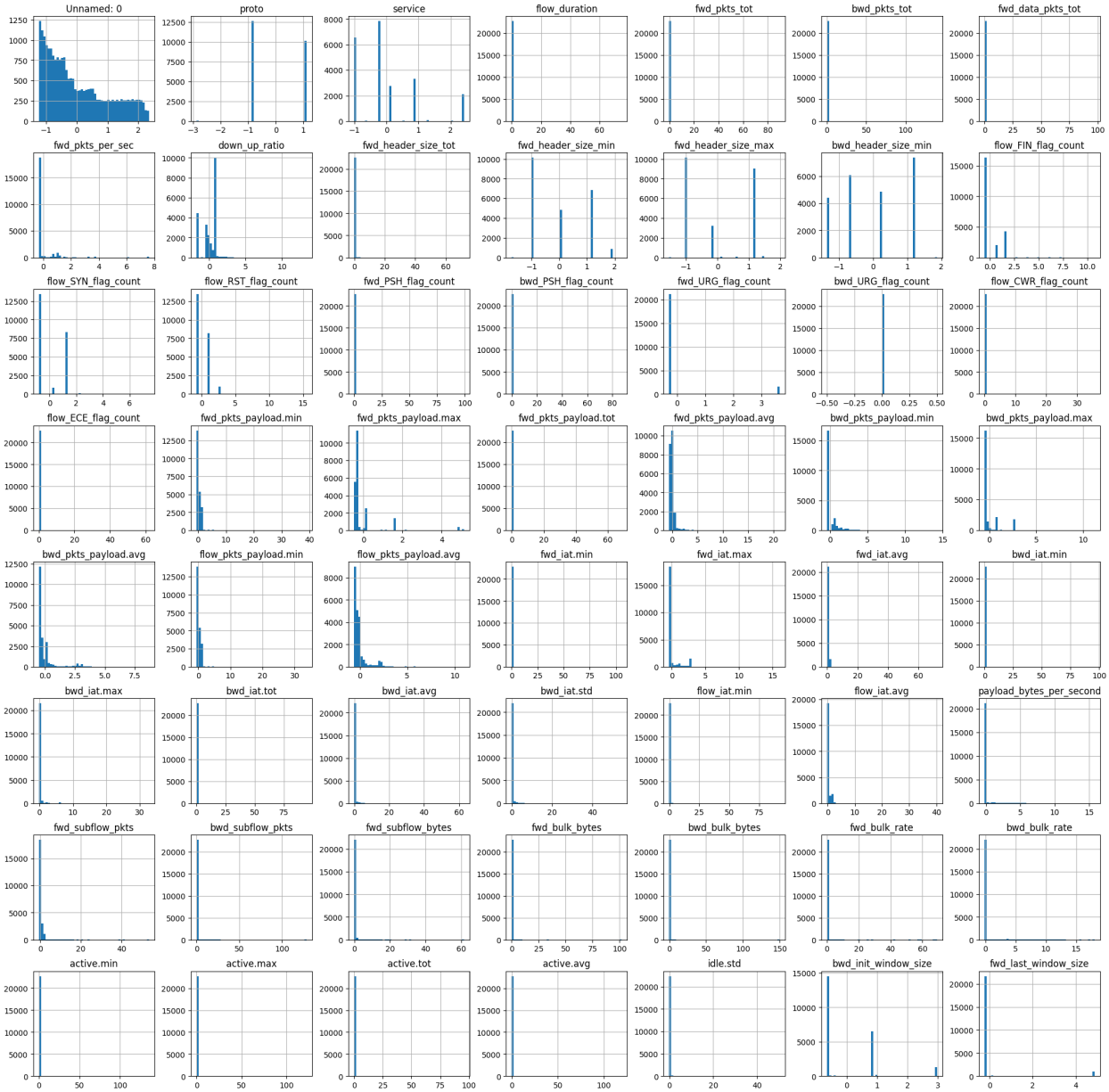
3.2.5. Train/Test Split with Stratification

Train/Test split is a fundamental practice in machine learning that increases model performance and reliability as it helps assess how well a model performs on unseen data and prevents overfitting. As part of the pre-processing stage for the project, the dataset was split into 80% for training and 20% for testing, using the stratify parameter to preserve class proportions; this ensured evaluation scores on minority classes are fair and prevented distribution drift.

3.2.6. Feature Scaling

When carrying out the data exploration phase, it was found that there was a need to implement scaling of the dataset for feature range normalization in order to remove magnitude differences, as it avoids the dominance of large magnitude features and enhances the performance for the base model as well as for *SVM* by allowing fair contribution of all features during training. The histogram grid below was plotted after using *StandardScaler* to transform the features in both the train and test splits. The figure shows that the features now have similar ranges centered around 0, that distributions remained heavily skewed with long tails which is expected in network data; and that outliers remain visible which was kept intentionally as they represent attack behavior.

Feature Distributions AFTER Scaling



3.2.7. Handling Imbalanced Data with SMOTE

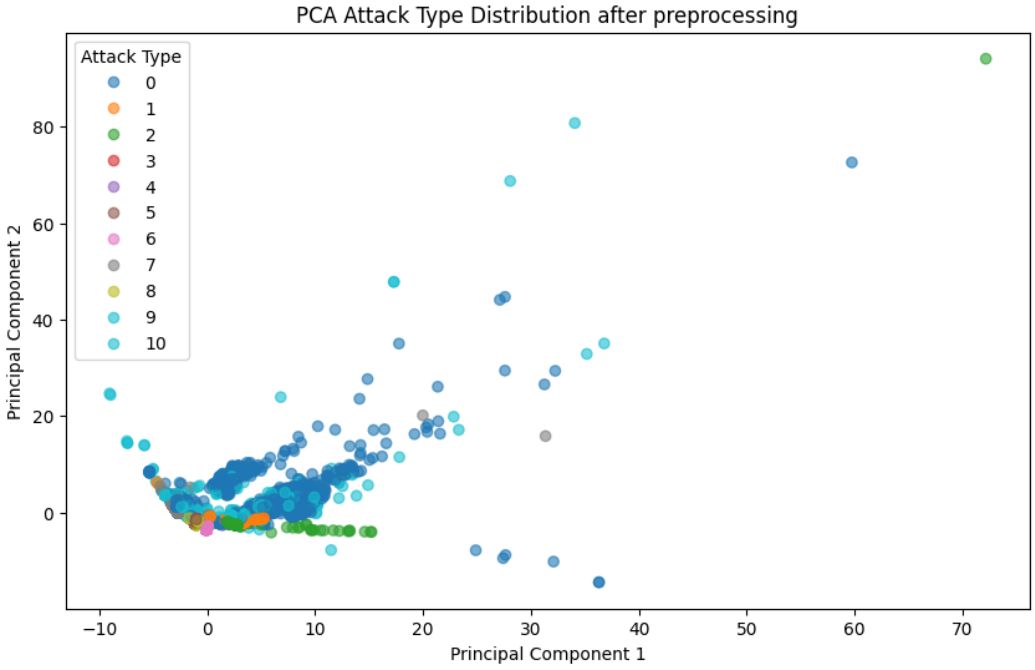
After removing the most dominant category during the initial cleaning of the dataset, the class imbalance persisted with an uneven distribution. Handling class imbalance was essential to prevent a biased classifier model that performs well on frequent attacks but fails to detect rare malicious behaviors (Musthafa y cols., 2024), for this reason and to enhance the precision of the dataset (Musthafa y cols., 2024), the resampling strategy with The synthetic minority oversampling technique (SMOTE) was applied only on the training set after scaling to avoid data leakage by not oversampling the test set. The results of this step can be seen when comparing the counts of the *y_train* (before SMOTE) and *y_train_balanced* (after SMOTE) sets are displayed in the table below.

Class #	Before	After
0	6200	6486
1	427	6486
2	3317	6486
3	30	6486
4	22	6486
5	1600	6486
6	802	6486
7	2072	6486
8	1608	6486
9	6486	6486
10	202	6486

3.3. Dimensionality Reduction with PCA

Once all pre-processing steps were implemented, another PCA visualization was performed to inspect the distribution of the target in the dataset. PCA can improve model efficiency and reduce noise by transforming correlated features while preserving variance (Almaiah y cols., 2022).

The plotted PCA shows that after the pre-processing, the samples now form tighter class clusters with less dominance of majority classes and a clearer separation for some of the categories (such as class 2). This confirms that pre-processing did improve the structure of the dataset however, many of the classes still overlap making accurate classification challenging for the base model, further validating the choice of training the more advanced models to capture non-linear feature interactions.



3.4. Model Development and Hyperparameter Optimization

With the data exploration and pre-processing phases finished, the training of multiple supervised learning models was carried out. The modeling strategy included the training of the baseline model (*Logistic Regression*) and continuing with the more advanced classifier models (*Random Forest* and *SVM*), which were evaluated first with their default parameters and then optimized using *GridSearchCV* for hyperparameter tuning. Finally, an ensemble *VotingClassifier* model, which combined all the selected models, was introduced to combine their strengths and improve overall performance.

3.4.0.1. Logistic Regression as Baseline Model

Because the problem at hand focuses on multiclass classification tasks, the baseline model chosen for the project was Logistic Regression as it is fast, easy to interpret and overall has a good reputation of performance for this type of cases. The model was trained on scaled and balanced data and evaluated using *Accuracy*, *F1-Score*, *Confusion Matrix* and *ROC-AUC*.

The hyperparameter tuning for Logistic Regression was done using a parameter grid that had the options 0.1, 1 and 10 for the *C* parameter; 300, 500 and 1000 for the *max_iter* parameter and *lbfgs* and *liblinear* for the solver parameter.

3.4.0.2. Advanced Models

With the purpose of addressing some of the limitations present in the baseline model, more expressive and non-linear classifiers were considered and introduced into the project. Prior studies have shown that *SVM* and *Random Forest* could deliver strong performance on IoT intrusion detection tasks (Samantaray y cols., 2024). These advanced models aim to improve the detection of minority attack classes and capture more complex decision boundaries in the feature space. The specific reasons to choose these two algorithms are:

- The *Random Forest Classifier* is commonly used in classification tasks like the one presented in this project. It uses the combined results of numerous classifiers to get more reliable predictions (Samantaray y cols., 2024) and contrary to the baseline model, it handles non linear structure better, is resistant to multicollinearity and provides feature importance insights. This model was trained on the scaled data, and the hyperparameter tuning was performed in *n_estimators* with the options 100 and 200, *max_depth* with the options *None* and 15 and *min_samples_split* with the options 2 and 5.

- The *Support Vector Machine* is similarly used for both classification and regression tasks as it is particularly effective for classification problems. Non-linear classification is possible using support vector machines thanks to the kernel approach for mapping inputs onto high-dimensional feature spaces excellent for overlapping and high-dimensional classes such as the ones present in our dataset (Samantaray y cols., 2024). The model was trained with the scaled data and the hyperparameter tuning with *GridSearchCV* was performed over C and Y, with key parameters *C* with options 0.1, 1 and 10; *kernel* with the value *rbf* and the parameter *gamma* with the options *scale* and *auto*.

3.4.0.3. Voting Classifier for Ensemble Learning

As all the previously selected models capture different properties of the attacks and have complimentary strengths and special characteristics, a popular approach instead of relying on a single model that has already been implemented in similar projects (Farooqi, Akhtar, Rahman, Sadiq, y Abbass, 2024), is aggregating the predictions of several classifiers by introducing a *Voting Classifier*, which determines the final output based on either majority voting or averaged probabilities. This ensemble model allows for the combination of strengths in order to prevent prediction failure (Farooqi y cols., 2024).

For the creation of this model, the *best_estimator* parameters that resulted from the hyperparameter tuning were used, and the voting type for the ensemble model was set for *soft voting* which gets the final prediction based on the average of predicted probabilities for each class.

4. Results and Evaluation

In this chapter the performance results of all trained models will be explained and evaluated in detail, in order to validate whether the methodology chosen to pre-process the dataset, train the models and their hyperparameter tuning helped solved the problem at hand and meets the project objectives. The models will be compared using a variety of metrics beyond just accuracy to ensure sensitivity to class disparities caused by the imbalance present on the data, as well as to improve interpretability of models' predictions. Additionally, multiple model prediction visualization analysis will be revised, so a deeper assessment of model's categorization performance can be achieved.

4.1. Evaluation Metrics

To ensure reliable evaluation of model performance, while being aware of the remaining class imbalance, multiple complementary metrics were used. While *Accuracy* indicates how often a model predicts the correct class (Musthafa y cols., 2024), in network intrusion detection this measure alone can be rather misleading in evaluating the model performance, as a model could classify most flows as majority classes and still reach a high score.

For this reason, *Precision* which highlights the ability to reduce false alarms and *Recall* which reflects how well the model correctly identifies positive classes (Musthafa y cols., 2024), are also considered. Since failing to identify attacks is more costly than simply issuing a false alert, *Recall* and *F1-score* also become key indicators of the effectiveness of the model to detect intrusions.

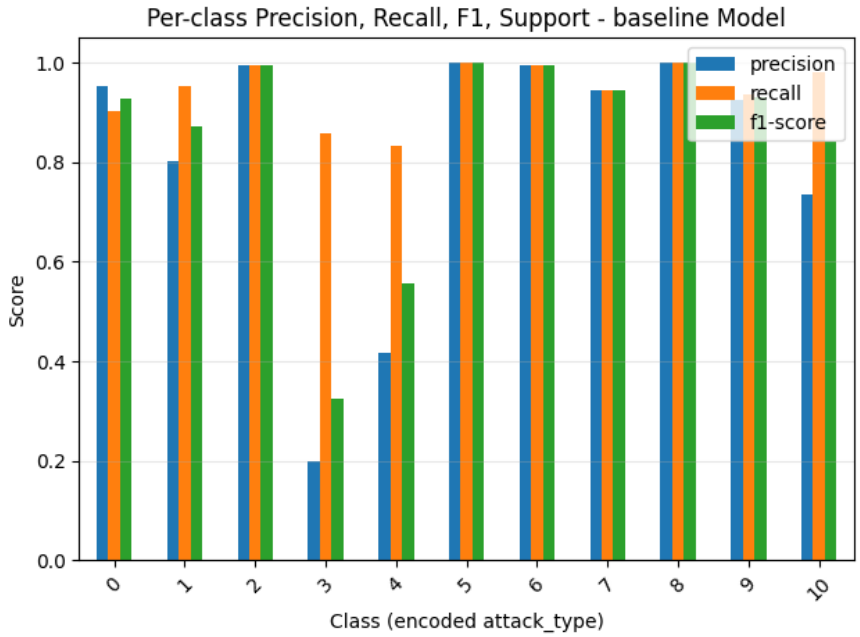
Additionally, *confusion matrices* are examined to identify weaknesses in predicting specific attack types putting emphasis on minority classes. Finally, *ROC-AUC* is applied in its multiclass configuration, allowing the observation of class separability and probabilistic confidence of the classifiers, meaning that a higher score implies stronger decision boundaries and better model generalization.

4.2. Model Performance

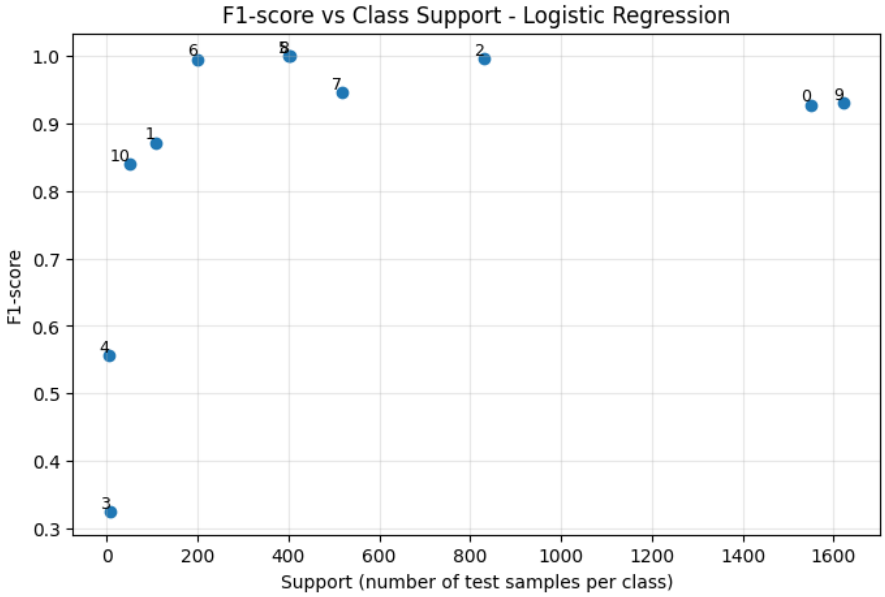
4.2.1. Baseline Model

The *Logistic Regression* algorithm as a baseline achieved overall a relatively high performance, making it a good starting point for the Machine Learning model. Its *accuracy* and *F1-score* both reached **95 %**, while the *ROC-AUC* scored pretty high with **0.992**, showing a great ability classifying high volume categories, hinting at their clear linear separability in the feature space.

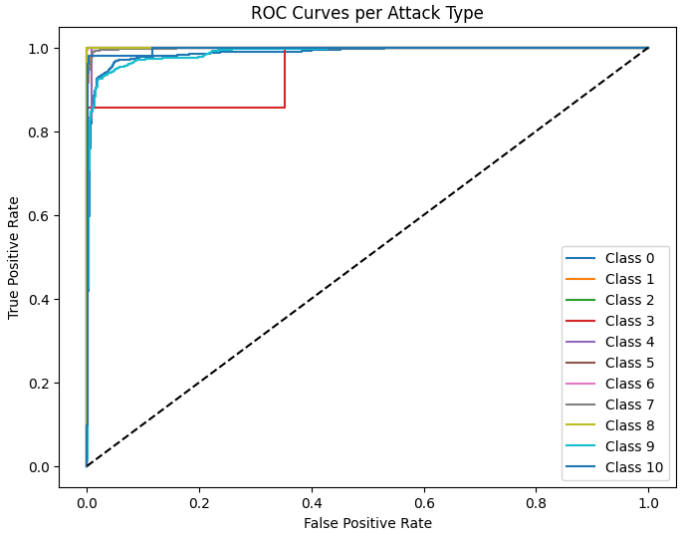
The visualized classification report below shows that attack classes with many samples achieve nearly accurate detection, while classes with few samples like class 3 and class 4 show high *Recall* and low *precision* and *F1-score*, indicating that the model struggles with minority class discrimination when the decision boundaries are overlapping.



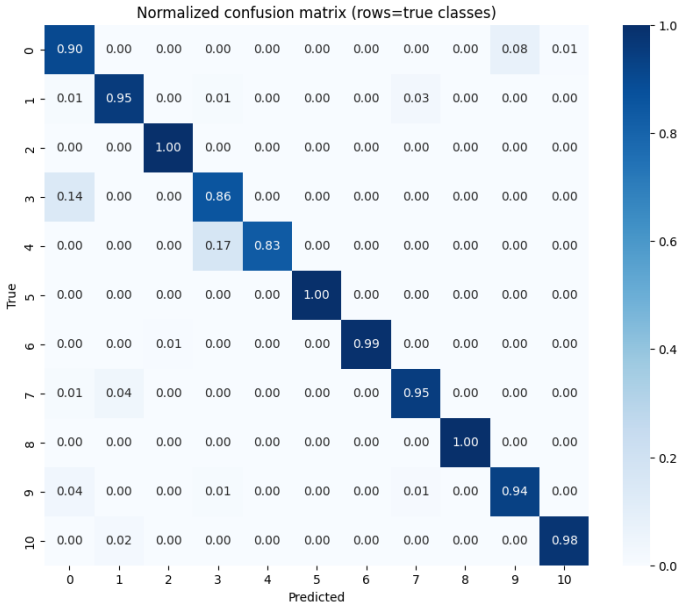
The *F1-score vs Class Support* plot below confirms that for the baseline model there is a strong correlation between the amount of samples per class during training, the better the F1-score, this makes minority classes have very low support and being rarely considered during predictions.



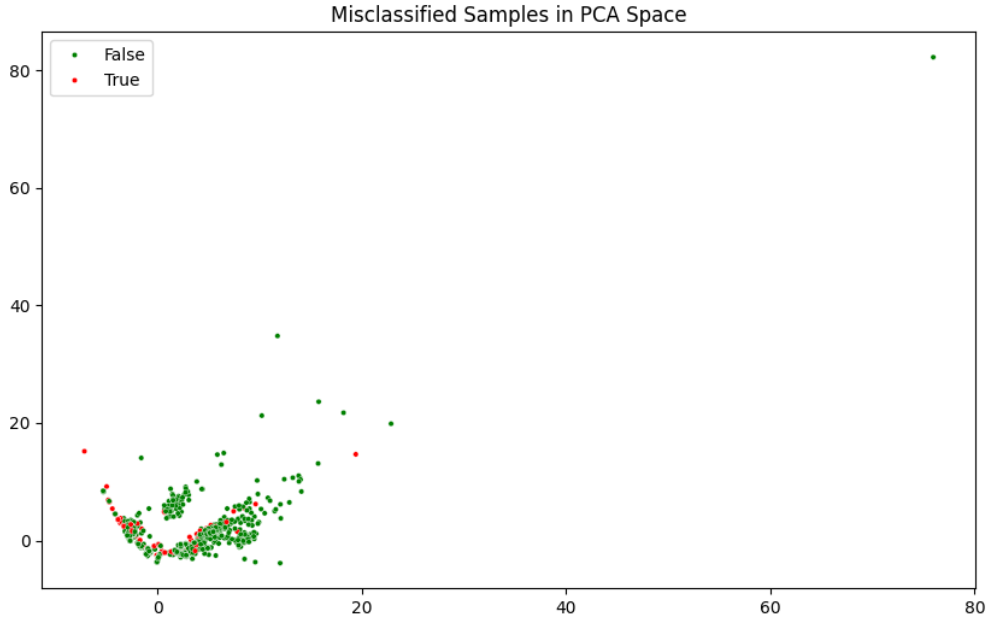
The visualized *ROC Curves* per category demonstrates most classes achieve a **99.3 %** AUC, confirming excellent rank ordering of predictions, however minority classes display fluctuations causing instability, as even though they are recognized, the model misclassifies them due to high class ambiguity.



The visualized confusion matrix below shows the classes 3 and 4 spread across the dominant classes 0 and 9, while approximately **4 %** of class 1 gets misclassified as class 7, and the dominant classes can sometimes also be confused with one another. These observations could mean that when classes occupy the same region in the feature space, the boundaries are not clear enough affecting accurate prediction likely caused by the model's reliability on linear separation.



The misclassification plots shown below confirms that the incorrect predictions are concentrated in regions where different attack classes overlap in feature space, indicating that the model's decision boundaries intersect areas of natural ambiguity between classes.



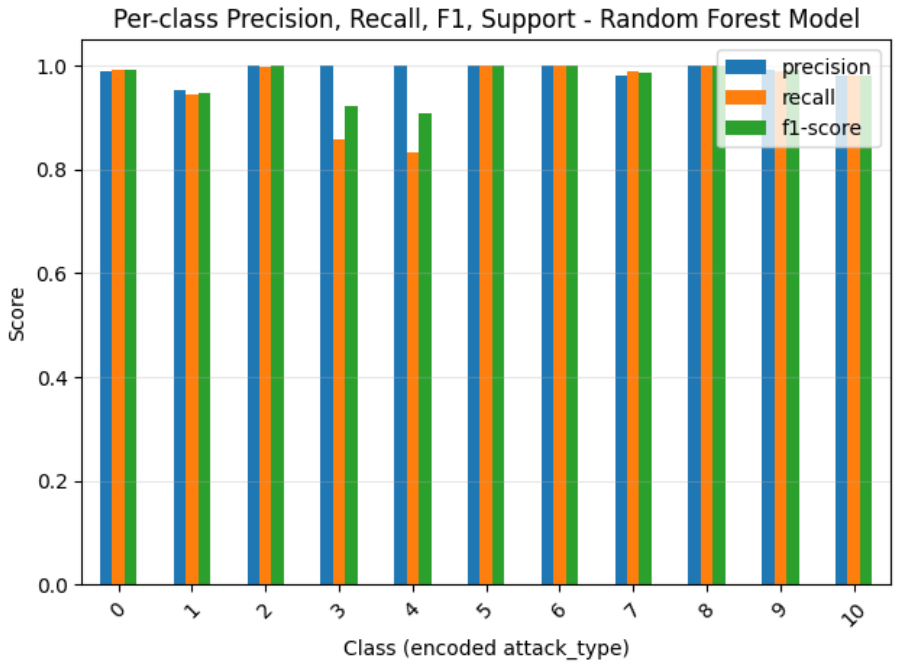
Overall, the *Logistic Regression* model demonstrates to be a very strong baseline, even considering that the algorithm is not necessarily suitable for the detection of rare network intrusions

4.2.2. Advanced Models Performance

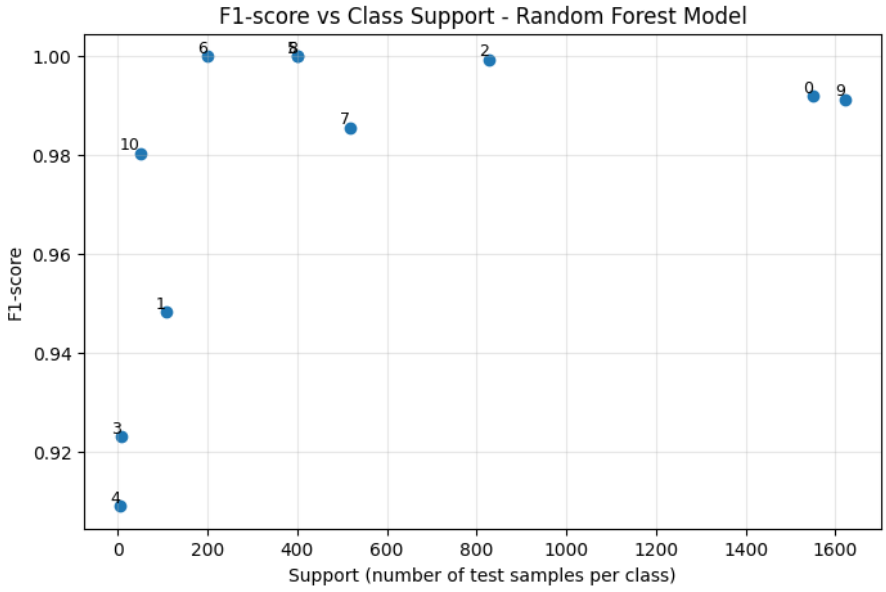
4.2.2.1. Random Forest Model

The *Random Forest* algorithm achieved the best performance among all of the models trained for the project, achieving an *accuracy*, *F1-score* of **99 %**, and a *ROC-AUC* of **0.992**, making it a very reliable at recognizing the different attacks on the dataset.

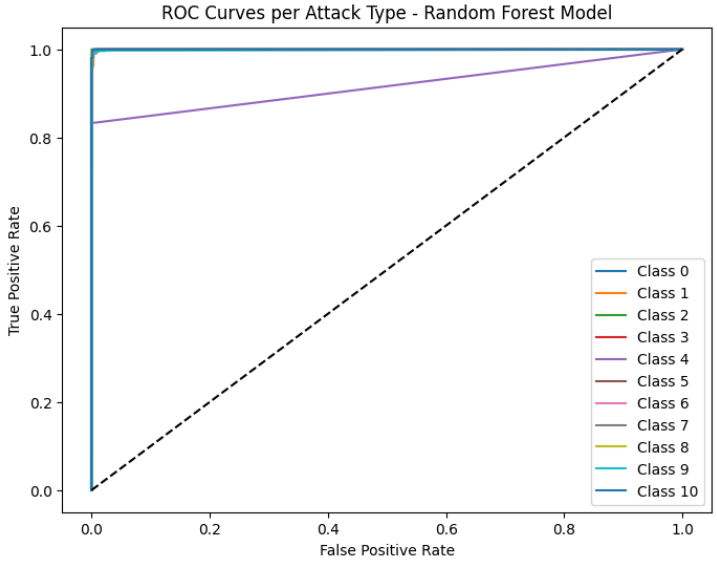
The classification report visualized underneath makes us understand that majority classes obtain an *F1-score* of at least **98 %** which means the classifier performs consistently on the most common attacks, while minority classes, in specific classes 3 and 4 which have a very small number of samples reach an *F1-scores* of **92 %** and *91 %* respectively, lower in comparison but remarkable given their support of less than 10 samples. These results reflect the fact that in comparison to the baseline model, the *Random Forest* is much less biased towards majority classes.



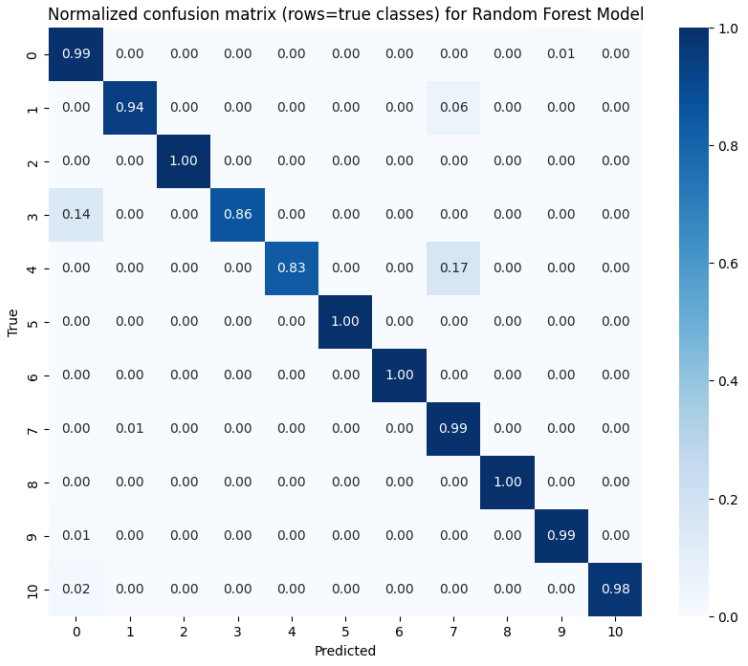
The visualized *F1 vs class support* plot below further proves that *Random Forest* remains superior in predicting minimal classes, and generalizing them well from minority oversampled data as even classes with a sample count of less than 15 can have a relatively high F1-score.



The plotted *ROC curves per class* display nearly perfect separation, where all *ROC curves* are in the top-left zone, which confirms low false positive rate (FPR) and high true positive rate (TPR). Even weakest class remains above *ROC-AUC* **0.83**.



In the plotted *Confusion Matrix* below, the diagonal contains nearly pure dark blue, confirming *Random Forest* predictions match ground truth with over **98 %** per class in most cases. There are still minor confusion between attack scans either because they are minority classes or there is overlap in network behavior, this could be interpreted as real network ambiguity rather than weakness of the model.

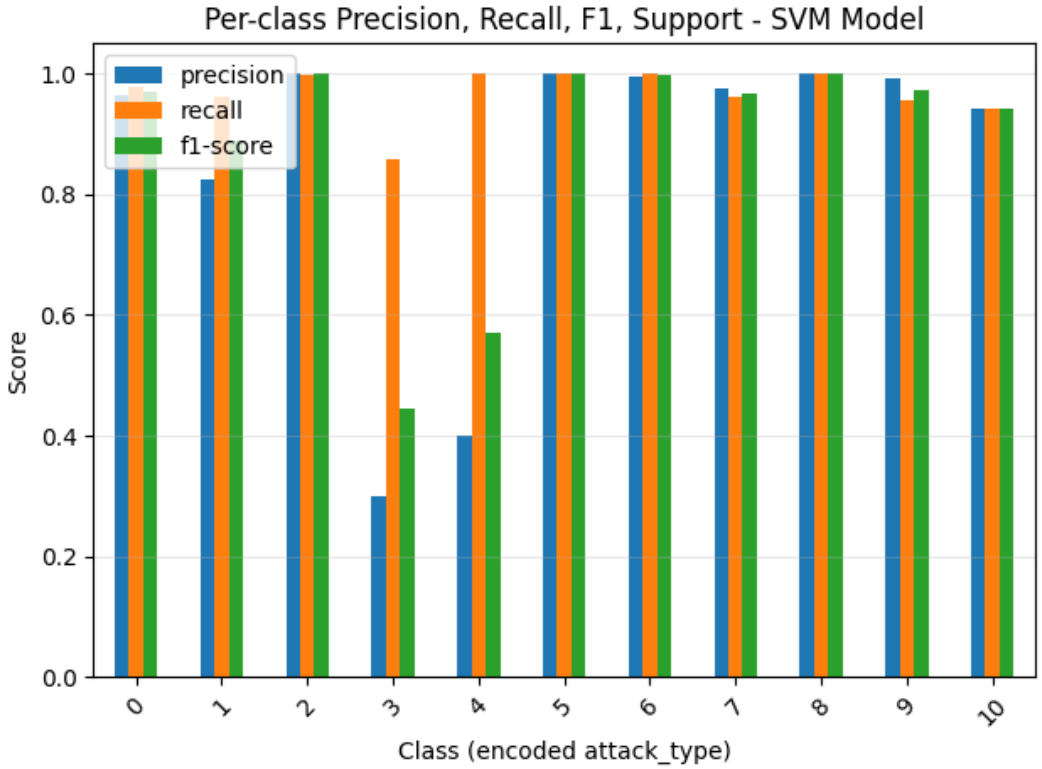


Overall, *Random Forest* showed best robustness against imbalance among all models, and remains the individual model with the best performance classifying attack types.

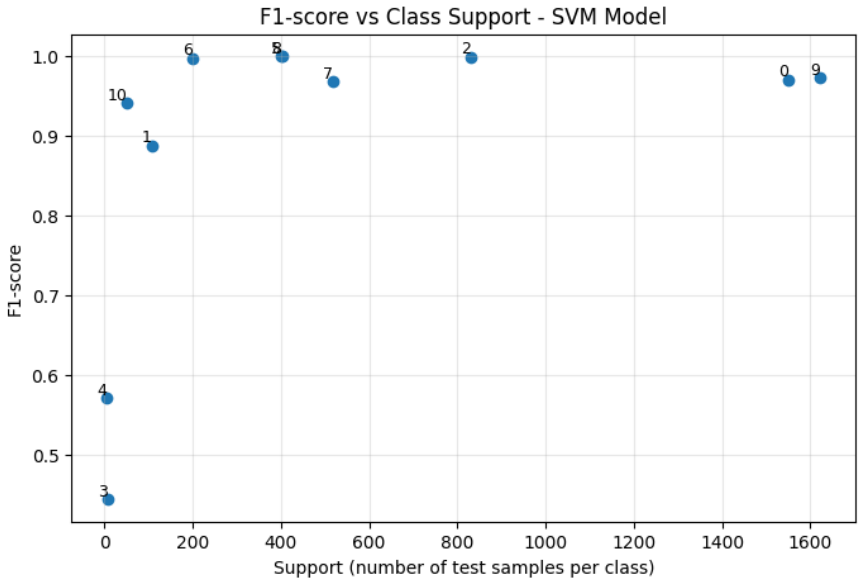
4.2.2.2. SVM Model

The *SVM* model reached an *accuracy* score of **97.6 %**, a *F1-score* of **98 %** and a ROC-AUC very close to *100 %* with **0.998**. This indicates excellent overall classification capability across the multiclass categorization problem, successfully distinguishing most attack types in the test set and effectively identifying and handling overlapping feature distributions.

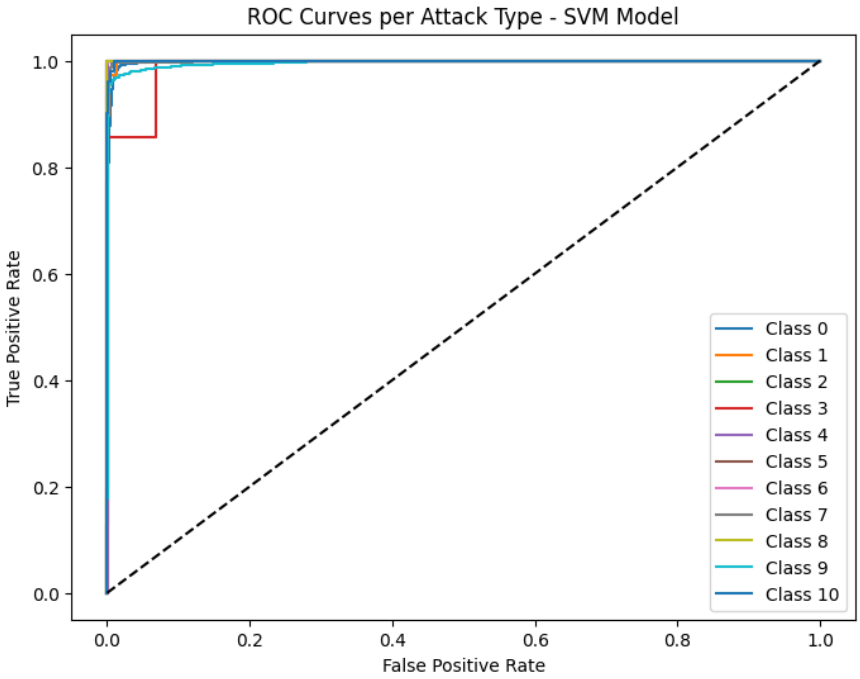
The visualized *Classification Report* below displays that similarly to the previous model, frequent attack types show almost perfect precision and recall, with F1-scores between **0.97** and **1.00**; while rare categories display significantly lower performance, often being confused or incorrectly labeled, like the minority classes 3 and 4 which have *F1-scores* of **44 %** and **57 %** respectively.



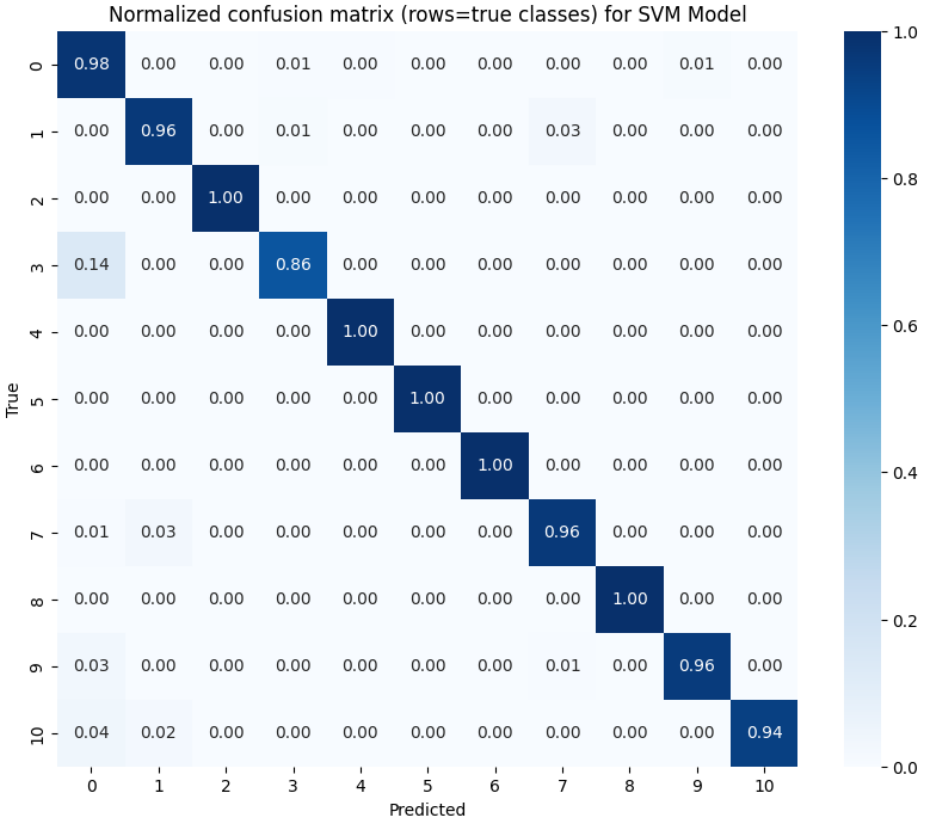
The plotted *F1 vs class support* diagram below, shows once more the clear correlation between the number of samples and the *F1-score* value. The differences between classes with over 500 samples and classes with less than 50, confirms that data scarcity, not necessarily the model itself, is the primary cause of misclassification.



The *ROC curves* displayed below show that most classes scored an *AUC* between **0.99** and **1.00**, with *AUC* slightly lower for minority classes but still high at around **85 %**, signaling that good separability despite noise and that classification is still significantly better than random guessing.



Finally, the plotted normalized *Confusion Matrix* shown below demonstrates that most predictions lie along the diagonal, signifying correct classification. However, the minority class 3 still contributes to the prediction mistakes. Additionally, there seem to be tiny spillovers towards classes 0 and 9 that could mean some attacks are being classified as non malicious network traffic, opening the door for potential false negatives.



This model offered excellent global metrics and demonstrated to be very strong at classifying attacks that are packet heavy and high volume. However, it fails to correctly classify low support classes, as well as it becomes sensitive to overlapping, it may under-detect rare but significant attacks, and for this reason, in a real world scenario additional anomaly detection measures may be required.

4.2.3. Voting Ensemble Performance

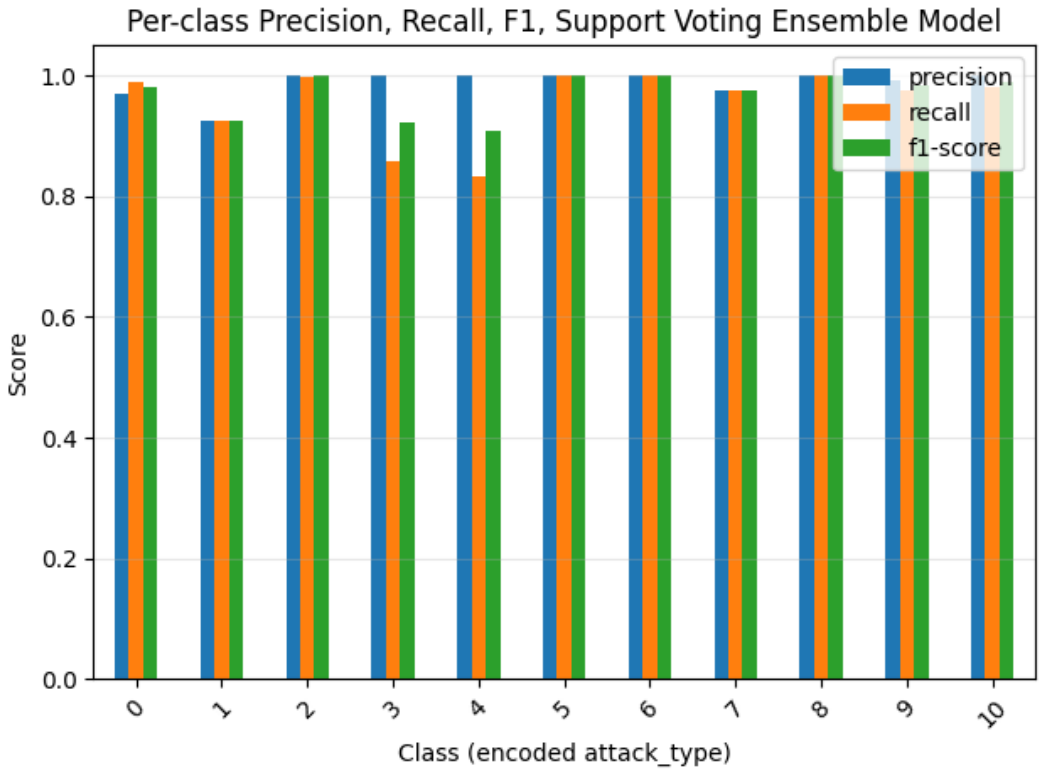
The final model evaluated was a *soft Voting Ensemble* combining the tuned *Logistic Regression*, *Random Forest*, and *SVM* classifiers. The objective behind using an ensemble method was to leverage the complementary strengths of each individual model to achieve higher prediction reliability, particularly for minority attack types which other models seems to consistently struggle with.

The ensemble uses soft voting with all tuned base models using *GridSearchCV* to find the models' best-performing hyperparameters:

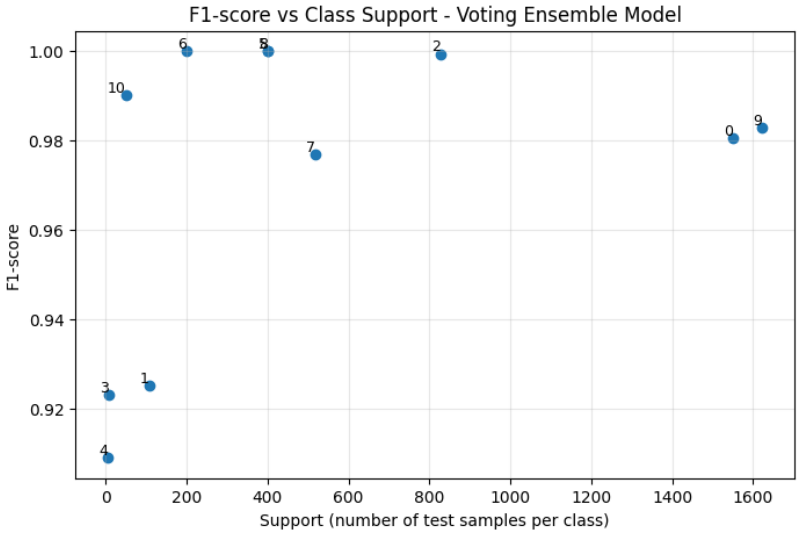
- For *Logistic Regression* the results were C=10, max_iter=500, solver='lbfgs'
- For *Random Forest* were n_estimators=200, max_depth=None, min_samples_split=2
- For *Support Vector Machine* the results showed C=10, gamma='auto', kernel='rbf'

The ensemble demonstrated strong performance across all the evaluation criteria, obtaining a **98.5 %** for *accuracy* and *F1-score* and a *ROC-AUC* score of **0.999** which shows the effectiveness of aggregating all the selected models for better performance.

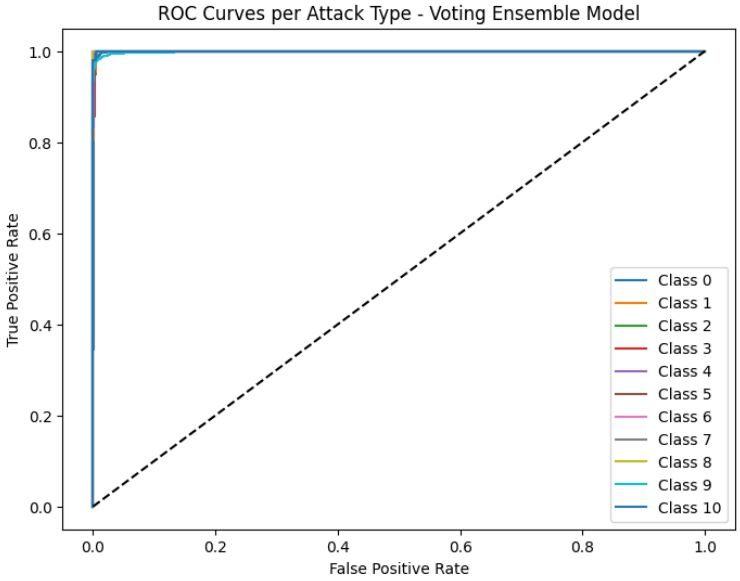
As shown in the visualized *classification report* below, the ensemble provides excellent consistency across precision, recall, and F1-score for almost all attack types. Majority classes all achieve scores above 0.98, indicating high detection reliability, while minority classes such as class 3 and class 4 maintain decent F1-scores around **0.91** and **0.93**. However, the results still show slightly reduced *recall* compared to other classes with better support. This suggests that while the ensemble has largely overcome the challenge posed by class imbalance, just like in the individual models, classes with low support remain difficult to perfectly identify.



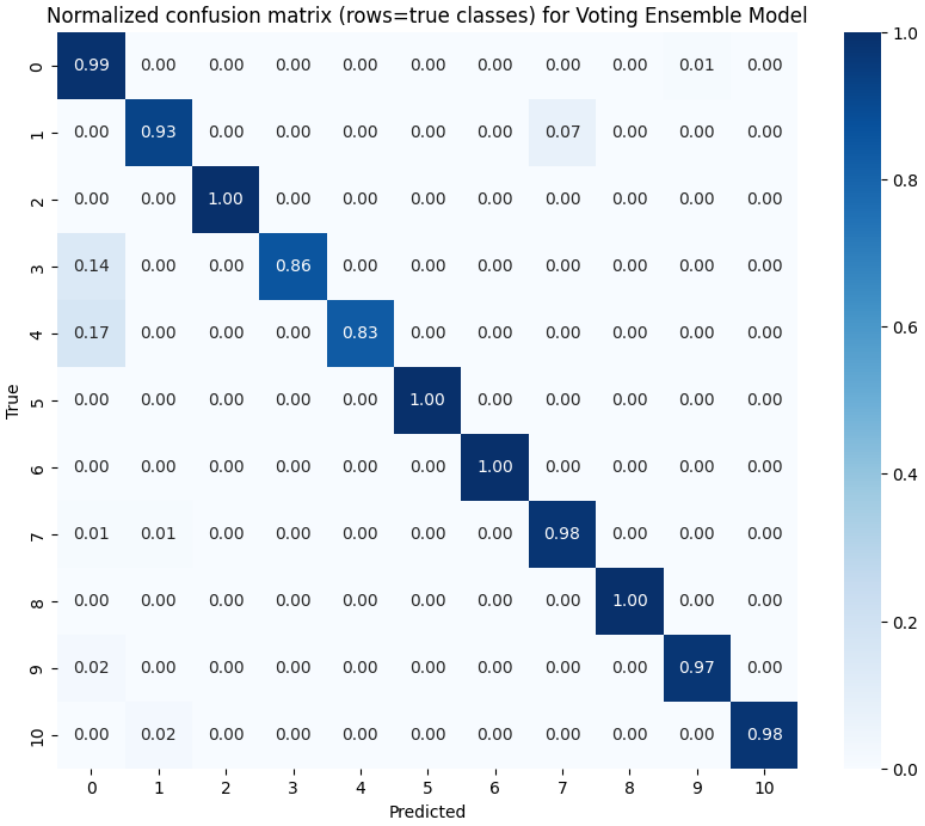
The *F1 vs support* plot visualizes how the relationship is strongly positive between high-support classes and almost perfect results, while the lowest-support classes sit around **0.91** and **0.93**. This has also been seen in previous models, however, it's worth mentioning that it's not the case for all minority classes, as *class 10* has a total of 51 samples and still managed to have a *F1-score* of **0.99** the minority classes in the ensemble model outperform equivalent performance seen previously in Logistic Regression and SVM, highlighting the ensemble's improvement on minority class robustness.



The *ROC curves* for all eleven classes in the figure below, are tightly concentrated in the top-left corner, showing highly confident separability. The ensemble achieves an overall ROC-AUC of **0.99976** making it the highest among all evaluated models.



The *normalized confusion matrix* visualized below confirms that most samples are classified correctly, with the diagonal cells approaching **1.0** for all major classes and, once again, misclassification mainly occurring in the minority classes 3 (with **14 %**) and 4 (with **17 %**). This again links back to very low sample support for these classes in the test set, as well as possible network feature overlap.

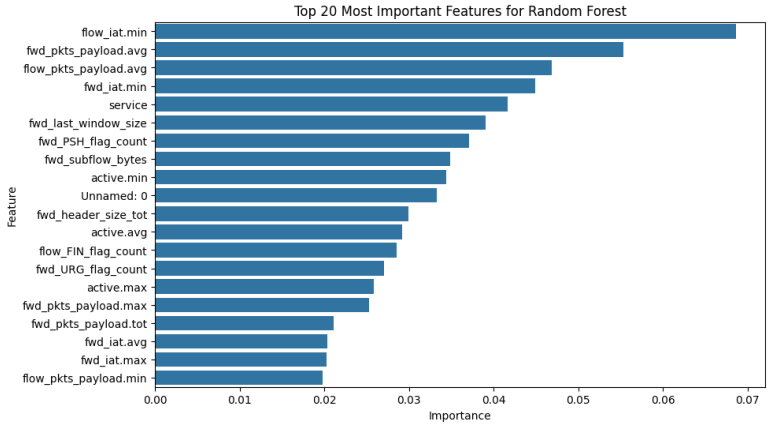


While the *Random Forest* model still exhibits slightly higher overall accuracy, the *soft Voting Classifier* slightly improved stability in predictions and minimized individual weaknesses introduced by the other models, offering the most reliable overall intrusion detection performance by displaying better generalization across all classes, making it the most balanced and practical model for deployment in a network intrusion detection environment.

4.2.4. Feature Importance Analysis

In order to better understand model decision-making and detect which network behaviors are most predictive of attack types, feature importance was analyzed for the two most interpretable models in this study *Random Forest* and *Logistic Regression*. The objective was to identify key traffic characteristics driving classification performance.

4.2.4.1. Random Forest Feature Importance

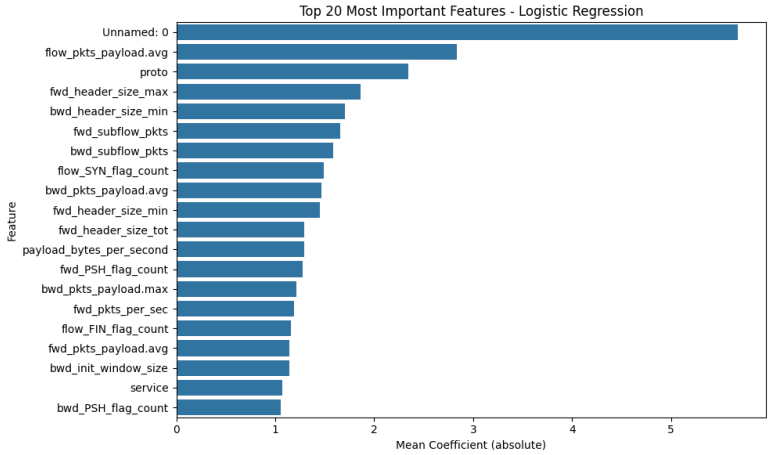


The figure above visualizes the Gini-based feature importance for *Random Forest* which shows that *flow_iat.min* (inter-arrival time minimum) is by far the most influential feature, indicating that rapid packet bursts are big signals for certain attacks.

Other important features are *fwd_pkts_payload.avg* and *flow_pkts_payload.avg* which relate to payload volume and consistency, *fwd_iat.min* and *fwd_iat.max* which are temporal characteristics of forward traffic, the feature *service* which is service level indicator; and some flag based attributes such as *fwd_PSH_flag_count* and *flow_FIN_flag_count*

It is possible to recognize a pattern across ranked features, that suggests that the model captures timing irregularities, payload burst patterns, and TCP control flag anomalies, which are commonly associated with specific attacks like *DDoS*, *brute-force*, and *port scanning*.

4.2.4.2. Logistic Regression Feature Importance



The figure above shows the linear measure of feature contribution based on the absolute coefficient values in Logistic Regression. Based on the figure it can be inferred that unlike *Random Forest*, this model relies more on transport layer structural features like *fwd_header_size_max*, *bwd_header_size_min* and *fwd_header_size_tot* which contain data on packet structure and negotiation, *fwd_subflow_pkts* and *bwd_subflow_pkts* which are flow segmentation behavior data; and *SYN/FIN/PSH* flag counts that helps indicate connection initiation and tear down.

The top-ranked feature *Unnamed: 0* is an index variable inherited from the dataset. Its unexpectedly high coefficient suggests that the model unintentionally exploited this artefactual pattern.

4.2.4.3. Feature Importance Comparison

After analyzing and then comparing linear and non-linear importance results, it can be affirmed that both models agree that flow timing and control flags are highly relevant in detecting attacks. *Random Forest* relies more on behavioral burst patterns, while *Logistic Regression* places more emphasis on packet structure characteristics.

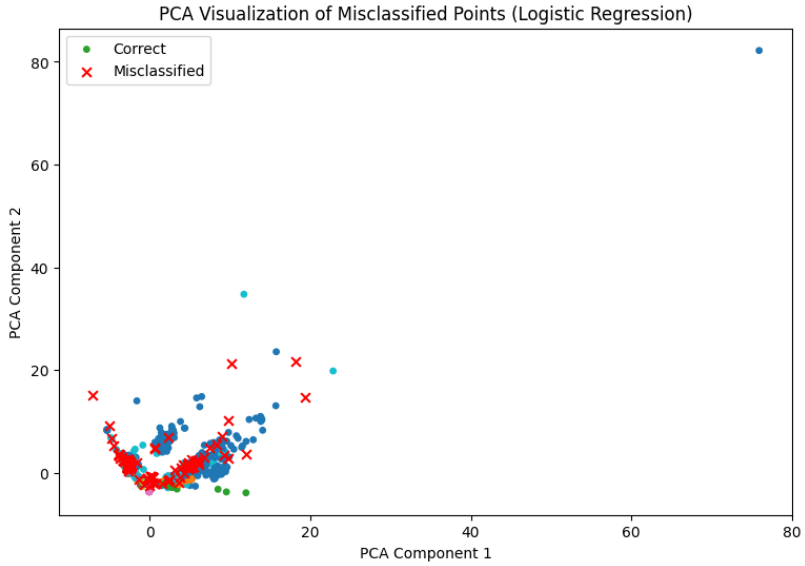
These complementary findings indicate that network cyber attacks exhibit both temporal burst anomalies and protocol-level structural irregularities, and effective intrusion detection benefits from models capable of capturing non-linear, multi feature interactions, such as *Random Forest*.

4.3. Misclassification Analysis using PCA

In an attempt to better understand model failure cases, PCA was used again to help project the high dimensional test data with which the models were trained into a two dimensional space. Correctly classified samples are shown in green and misclassified samples in red. This allows visual inspection of whether erroneous predictions occur in overlapping feature regions or in rare, isolated observations.

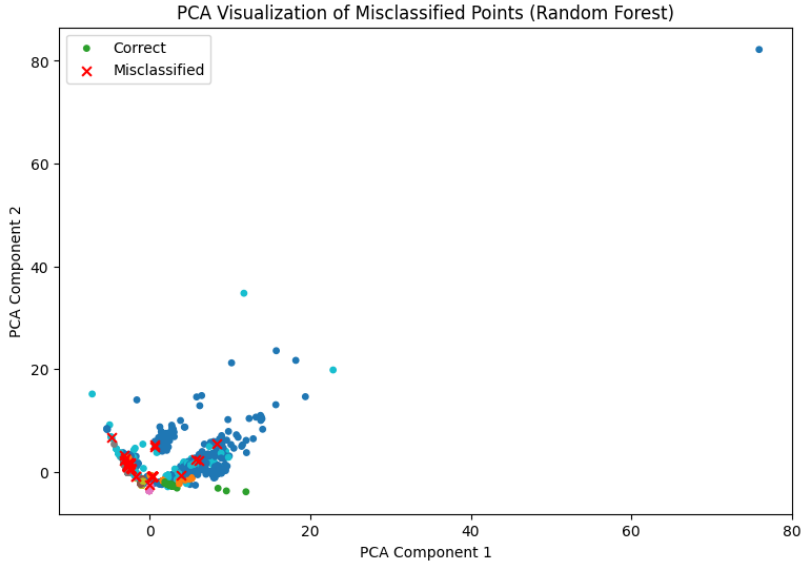
4.3.1. Logistic Regression Analysis

The figure below shows how *Logistic Regression* misclassifications were primarily located within dense regions of overlapping categories. supporting the conclusion that linear decision boundaries were not sufficient to completely separate complex traffic behaviors. The presence of multiple red markers within the core cluster suggests that attacks with similar features were frequently confused with one another. Only a few isolated samples at the edges of the PCA space were correctly classified, further demonstrates the baseline model's difficulty in handling non-linearity.



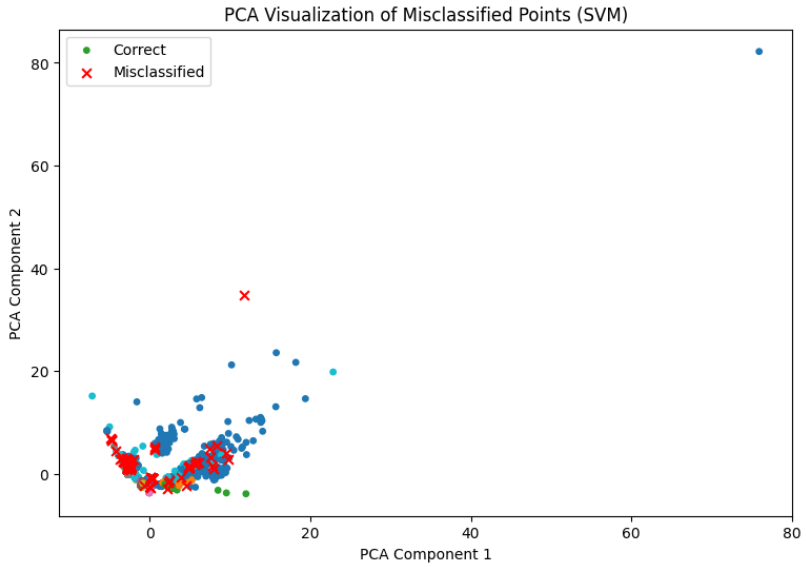
4.3.2. Random Forest Analysis

This model showed significantly fewer misclassifications, most red points in the figure below appear only at the sidelines of class clusters. This exemplifies how the model managed to capture complex feature interactions and decision rules more effectively than the baseline model. The other errors seen in the plot occur only where class distributions overlap in the projected space, but as explained previously, that might be more about the nature of network metadata rather than a model limitation.



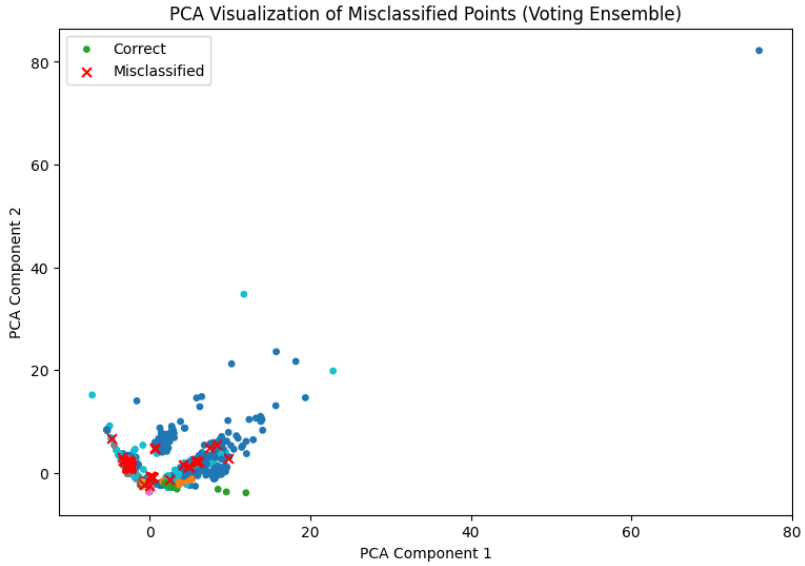
4.3.3. SVM Analysis

According to the PCA plot below, the *SVM* model produced a small number of misclassifications clustered within the main overlapping region. This again confirms that class confusion is caused largely from similarity in network features. Additionally, a notable outlier misclassification is visible far from all cluster centers, visualizing the model's sensitivity to extreme or low support samples in the training set. Despite this, the model handled non-linear boundaries substantially better than the base model.



4.3.4. Voting Classifier Ensemble Analysis

As represented in the figure below, the *Voting Ensemble* achieved the lowest number of misclassifications, with errors appearing only in a very reduced area of class overlap. This supports that combining all the selected models, was the right strategy to help resolve disagreements in ambiguous regions. Aside from that, just like in the *Random Forest* model, the remaining misclassified samples seem to be happening due to high feature similarity with neighboring classes and don't reflect weaknesses in the model.



Across the selected models, misclassifications seem to consistently originate from the same dense *PCA* region where multiple categories cluster together. This suggests that these samples share nearly identical flow characteristics, making them challenging even for advanced models. Fortunately, the approaches of the *Ensemble* model and the *Random Forest* model effectively mitigate these issues, while baseline model struggles with the non-linear nature of class boundaries found in the dataset.

4.4. Performance Comparison

It is imperative to compare the effectiveness of the evaluated models performance metrics in order to understand whether the decisions taken for the Machine Learning Project positively contributed to the solution of the problem at hand. After training the models, making predictions and visualizing the results, it was found that the *Voting Ensemble*, *Random Forest*, and *SVM* models achieved an almost perfect category classification, all exceeding **97 %** *accuracy* and *F1-score*, while Logistic Regression was left noticeably behind at approximately **95 %**. *Random Forest* obtained the highest overall *accuracy* (**99.26 %**), closely followed by the *Ensemble model* (**98.59 %**) and *SVM* (**97.64 %**).

The data used to test models was very imbalanced as proven during the data exploration phase, with some classes having less than 10 samples. The scatter plots of F1-score vs support showed that the baseline model and *SVM* struggled on rare attack types, however, **Random Forest** and the *Ensemble* maintained *F1-scores* higher than **0.90**, even for minority classes and also achieved perfect recall for several attacks, showing excellent threat detection reliability. This demonstrates that ensemble approaches are more resilient to low support categories, which is a highly impor-

tant characteristic in real world intrusion detection systems, where rare attacks are often the most critical to detect.

It could be said that the models can be separated between two groups, *Logistic Regression* and *SVM* which both had big gaps between *precision* and *recall* for minority classes, and *Random Forest* and the *Voting Ensemble* which maintained a good balance between said metrics, with fewer trade-offs, as well as stronger reliability and interpretability.

Upon comparing the results of the selected models it can be concluded that *Random Forest* and the *Voting Ensemble* are the most effective models for detecting and classifying attack types in the used dataset. Their superior ability to learn non-linear relationships and keep high *recall* on rare categories makes them best suited for deployment in real intrusion detection systems. *Logistic Regression* as a baseline model remains efficient, but upon deeper analysis, it should not be used as the primary detection engine due to significant boundary misclassification problems.

5. Conclusions

5.1. Discussion

The execution of this project involved a big learning curve and multiple trial and error situations that ultimately shaped the path to solve the project's problem and guided the methodology decisions for the right implementation of the machine learning model. Once the model implementation, training and predictions were done and the results properly analyzed, there are a few takeaways to be acknowledged so they can serve as considerations when creating a similar Machine Learning Model.

5.1.1. Strengths of the Final System

These strengths reflect a strong alignment between the data characteristics and the chosen modeling strategies.

- The high *recall* on multiple attack types when predicting with the ensemble models (*Random Forest* and *Voting Classifier*) considerably aids intrusion detection by reducing the risk of undetected threats.
- The strength against imbalance datasets in the ensemble models further supports intrusion detection by having a decent accuracy when classifying minority classes.
- Allowing model interpretability through feature importance analysis for the baseline and *Random Forest* models, gives a clear idea of how the classification decisions were made and exactly how relevant to prediction each feature in the dataset is.
- Both the *Random Forest* and *Logistic Regression* classifiers show strong deployment feasibility since after training, they start performing classification tasks with minimal latency, making them practical for real time intrusion detection systems.

5.1.2. Limitations

Despite strong overall performance several challenges are left to be addressed on this project, but are worth mentioning so they can be taken into consideration when working on similar projects.

- Cyber attack patterns and IoT behaviors continuously change in production environments, and although the models shown high performance in classifying attacks, simplified

models often lack the flexibility to adapt to evolving attack types (Sharma y cols., 2025), meaning the model would require periodic retraining to remain accurate.

- Despite strong global results, some misclassification persists between similar attack types, particularly in minority classes, demonstrating that classification for this classes remains challenging.

- Low support categories such as *Class 3* and *Class 4* still form overlapping clusters in PCA space, suggesting that these attacks have strong feature similarities that conventional models struggle to separate.

- All experiments were conducted on a historical dataset, however, effective intrusion detection must operate continuously and adapt to new and unseen threats that were not present during training time.

- While SMOTE improved performance metrics, overfitting cannot perfectly reflect real malicious behavior and can also bias decision boundaries.

5.2. Conclusion

The results of this project demonstrated that Machine Learning provides an effective approach for real time intrusion detection in IoT infrastructure. The multiclass classification experiments carried out, confirmed that while linear methods such as *Logistic Regression* established a strong baseline, they can struggle to fully capture complex and non-linear structures typically seen in malicious IoT traffic.

By applying careful data pre-processing, correlation based feature reduction, oversampling with SMOTE and hyperparameter tuning, the performance of all selected models improved considerably. *Random Forest* and *SVM* showed superior robustness in differentiating subtle differences among attacks, with the *Random Forest* model achieving the highest overall accuracy and reliability across all classes.

In order to leverage the different strengths of the chosen models, a hyperparameter tuning with *GridSearchCV* was done to the selected models and a soft-voting ensemble was also implemented, combining *Logistic Regression*, *Random Forest* and *SVM* classifiers. This ensemble achieved the most balanced results, correctly identifying low support attacks while maintaining excellent generalization on the test set, making it the best candidate for deployment in IoT intrusion detection scenarios.

However, although performance was high, several open challenges remained, for example, minority attack types exhibit overlapping behavior that still causes occasional misclassification and

the static offline evaluation does not yet account for evolving real world threat landscapes. Future work should therefore focus on dynamic model retraining, anomaly detection for zero day attacks, and deeper interpretability which is crucial in cybersecurity to understand why a model flags traffic as malicious and to support trust in automated systems (Sharma y cols., 2025).

Finally, this project did manage to successfully meet its objective to build a supervised Machine Learning model capable of classifying malicious IoT traffic with high precision and practical deployment potential, that could potentially be deployed into a real time intrusion detection system.

Referencias

Almaiah, M. A., Almomani, O., Alsaaidah, A., Al-Otaibi, S., Bani-Hani, N., Al Hwaitat, A. K., ... Aldhyani, T. H. H. (2022). Performance investigation of principal component analysis for intrusion detection system using different support vector machine kernels. *Electronics*, *11*(21), 3571. Descargado de https://www.mdpi.com/2079-9292/11/21/3571 doi: 10.3390/electronics11213571

Farooqi, A. H., Akhtar, S., Rahman, H., Sadiq, T., y Abbass, W. (2024). Enhancing network intrusion detection using an ensemble voting classifier for internet of things. *Sensors*, *24*(1), 127. Descargado de https://www.mdpi.com/1424-8220/24/1/127 doi: 10.3390/s24010127

Musthafa, M. B., Huda, S., Kodera, Y., Ali, M. A., Araki, S., Mwaura, J., y Nogami, Y. (2024). Optimizing iot intrusion detection using balanced class distribution, feature selection, and ensemble machine learning techniques. *Sensors*, *24*(13), 4293. Descargado de https://www.mdpi.com/1424-8220/24/13/4293 doi: 10.3390/s24134293

Real-time iot2022 cyber attack dataset. (s.f.). Descargado 2023, de https://www.kaggle.com/datasets/joebeachcapital/real-time-internet-of-things-rt-iot2022/data (Kagle Dataset)

Samantaray, M., Barik, R. C., y Biswal, A. K. (2024). A comparative assessment of machine learning algorithms in the iot-based network intrusion detection systems. *Decision Analytics Journal*, *11*, 100478. Descargado de https://www.sciencedirect.com/science/article/pii/S2772662224000821 doi: 10.1016/j.dajour.2024.100478

Sharma, K. P., Nagpal, T., Vora, T., Yadav, A., Abdullah, M. I., Jayaprakash, B., ... Bukate, B. B. (2025). Interpretable intrusion detection for iot environments using a self-attention-based explainable ai framework. *Scientific Reports*, *15*, 39937. Descargado de https://www.nature.com/articles/s41598-025-23750-0 doi: 10.1038/s41598-025-23750-0